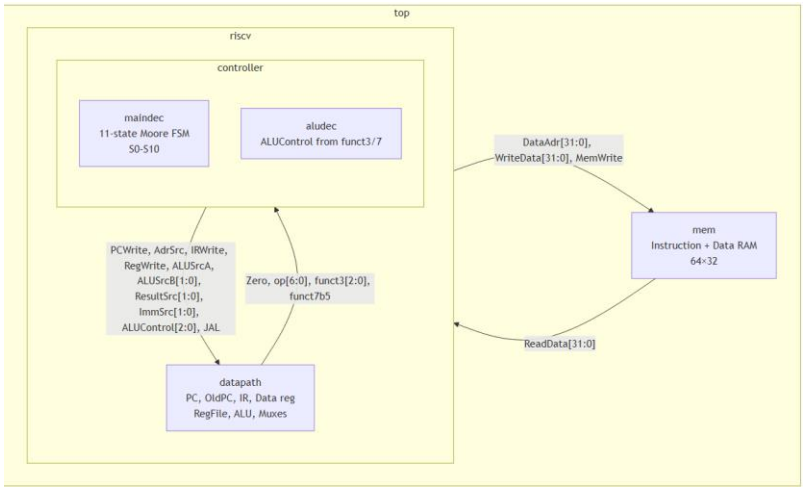
ELE432 – Lab 3: Multicycle RISC-V Processor

Taha Enes Erdem 2200357093

1. Module Hierarchy and Signal Diagram

The multicycle processor is organized into the following hierarchy:



Signals between riscv and mem:

- DataAdr [31:0]: memory address (PC during Fetch, ALUOut during Mem access)
- WriteData [31:0]: data to write (from RD2 pipeline register)
- MemWrite:write enable
- ReadData [31:0]: data read from memory (feeds both IR and Data register)

Controller to Datapath signals:

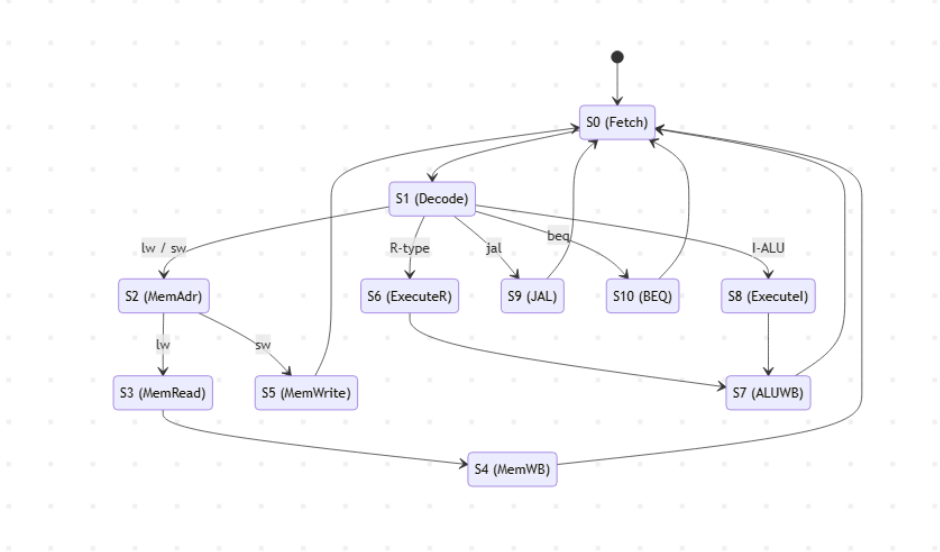
PCWrite, AdrSrc, IRWrite, RegWrite, ALUSrcA, ALUSrcB[1:0], ResultSrc[1:0], ImmSrc[1:0], ALUControl[2:0], JAL

Datapath -> Controller signals:

Zero, op[6:0], funct3[2:0], funct7b5

2. FSM State Encoding

The main decoder implements an 11 state Moore FSM. State transitions:



3. Key Design Decisions

SrcA override in Fetch: During S0, SrcA = PC (not OldPC) so ALU correctly computes PC+4. In all other states, SrcA = OldPC (when ALUSrcA=0) or A (when ALUSrcA=1).

JAL handling: In S1 (Decode), ALU pre-computes OldPC+ImmJ -> ALUOut (the jump target). In S9 (JAL state), ALU recomputes OldPC+4 (ResultSrc=10 -> Result=ALUResult = return address). RegWrite=1 writes return address to rd. JAL=1 overrides PC <- ALUOut (the jump target), bypassing the Result mux.

SLT (signed less-than): Implemented using overflow-aware formula: slt = sign(a−b) XOR overflow, where overflow = (a[31]^b[31]) and (a[31]^result[31]).

4. Table 1 – Expected Operation (Simulation Trace)

State names: S0=Fetch, S1=Decode, S2=MemAdr, S3=MemRead, S4=MemWB, S5=MemWrite, S6=ExecuteR, S7=ALUWB, S8=ExecuteI, S9=JAL, S10=BEQ

Step	PC	Instr	State	ALUResult / Result	Notes
3	00	00500113	S0: Fetch	00000004 (PC+4)	IRWrite; OldPC<-0; PC<-4
4	04	"	S1: Decode	00000005 (OldPC+Imm)	OldPC+Imm=0+5; ALUOut<-5
5	04	"	S8: ExecuteI	00000005 (x0+5)	SrcA=x0=0, SrcB=5; ALUResult=5
6	04	"	S7: ALUWB	00000005 -> x2	RegWrite; Result=ALUOut=5; x2<-5
7	04	00C00193	S0: Fetch	00000008 (PC+4)	IRWrite; OldPC<-4; PC<-8
8	08	"	S1: Decode	00000010 (OldPC+Imm)	OldPC=4+12=16; ALUOut<-16
9	08	"	S8: ExecuteI	0000000C (x0+12)	SrcA=x0=0, SrcB=12; ALUResult=12
10	08	"	S7: ALUWB	0000000C to x3	x3<-12
11	08	FF718393	S0: Fetch	0000000C (PC+4)	IRWrite; OldPC<-8; PC<-0xC
12	0C	"	S1: Decode	FFFFFFF7 (OldPC+Imm)	OldPC=8+(-9)=−1; ALUOut<-−1
13	0C	"	S8: ExecuteI	00000003 (x3+(−9))	SrcA=x3=12, SrcB=−9; 12−9=3
14	0C	"	S7: ALUWB	00000003 -> x7	x7<-3

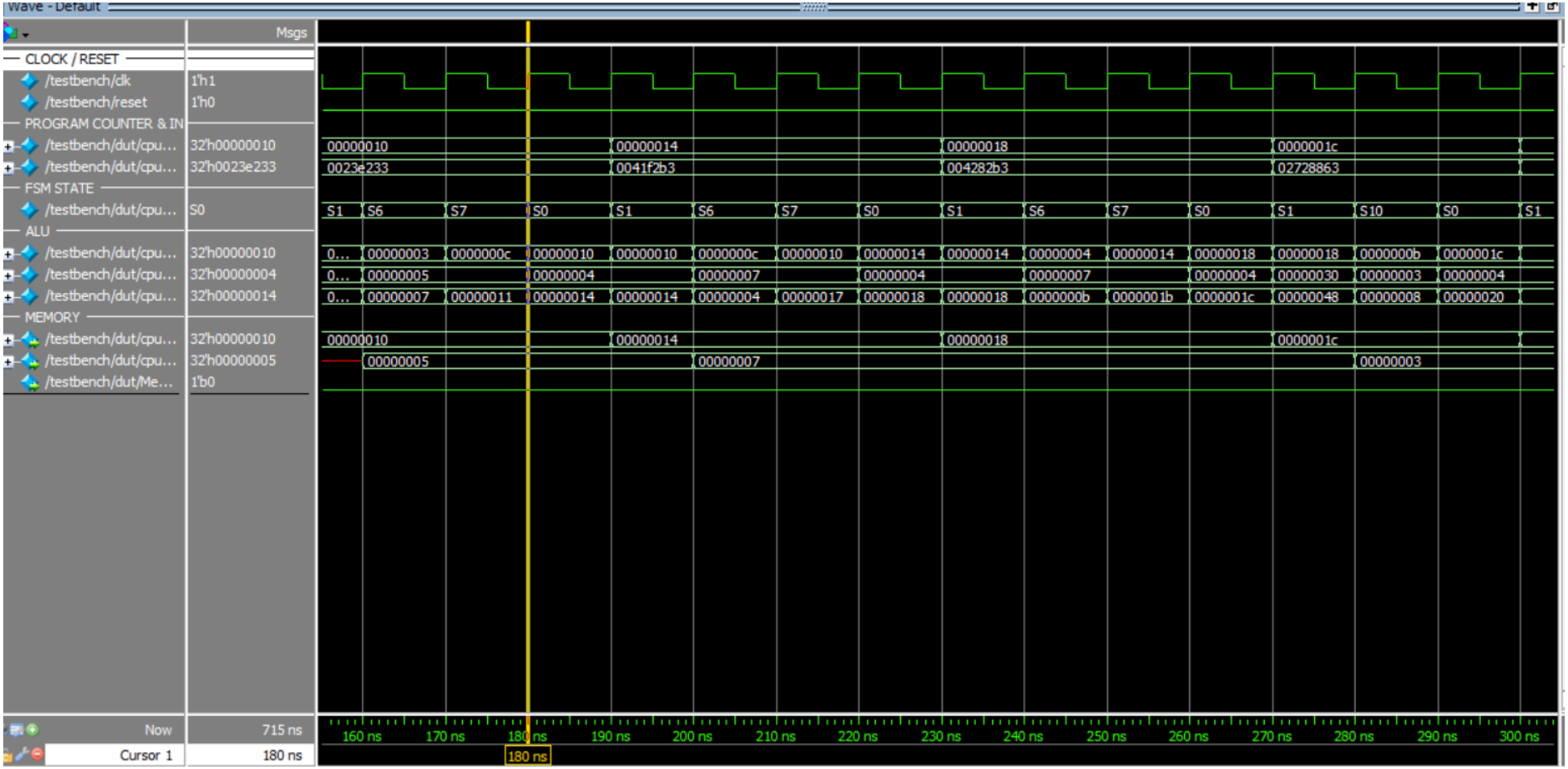
5. Testbench Result

The testbench checks for a successful write of WriteData=25 to DataAdr=100. The simulation completes with:

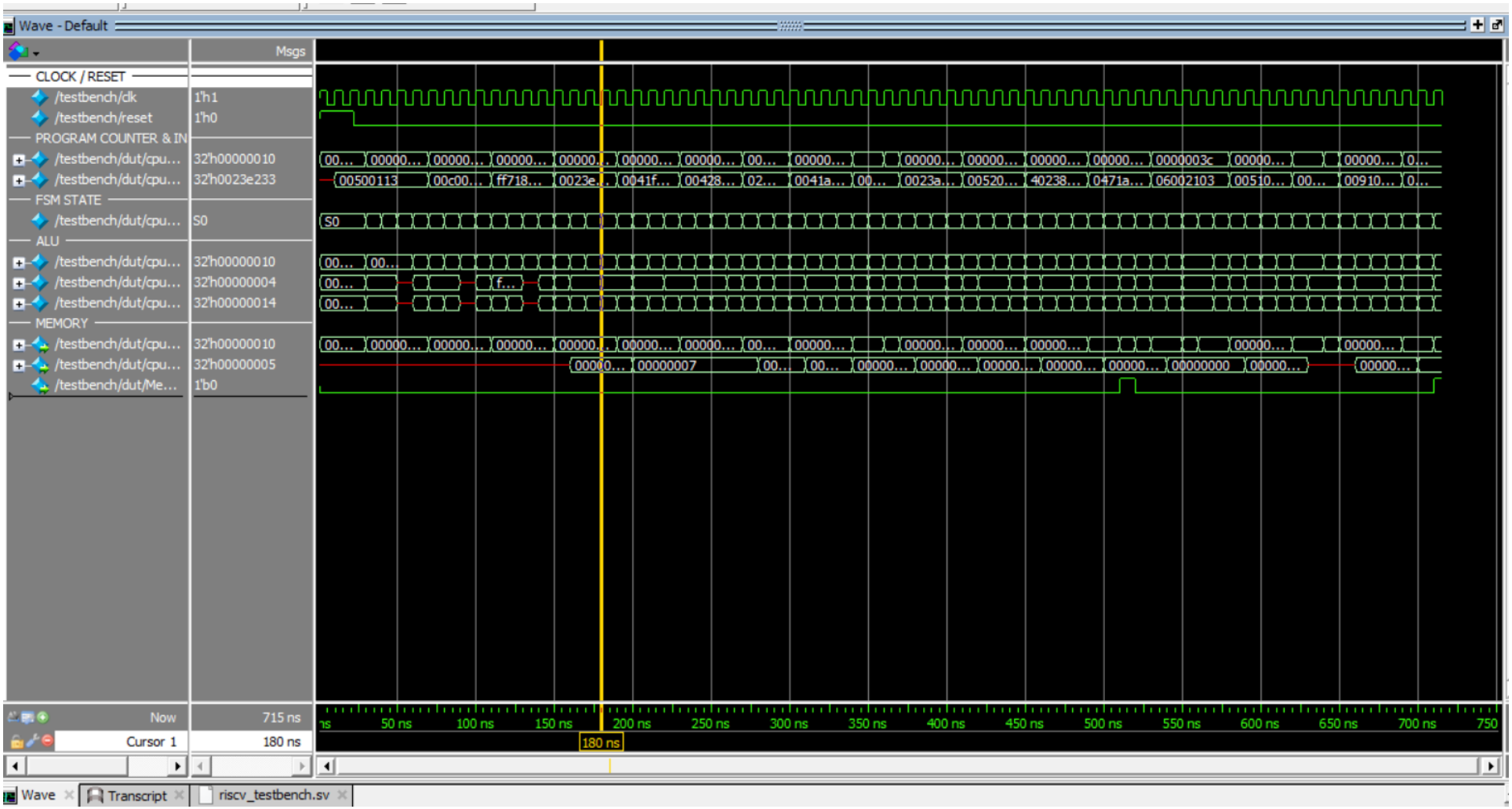
```
Simulation succeeded
```

This confirms that the processor correctly executes all instructions including addi, or, and, add, sub, slt, beq (taken and not-taken), sw, lw, and jal, and writes the correct result (25) to address 100.

Single cycle:



Full Sim:



7. Time Spent and GitHub Link

Approximately 6 hours (design, implementation, debug, and documentation).

GitHub link for this preliminary work is below:

https://github.com/tahaeneserdemm/ELE432pre-lab3